

MACHINE LEARNING

(ITY23PCL302)

UNIT 3 REGRESSION

Course Teacher: Ms.Priyanka A.Hajare

CONTENT

- Unit 3 : Regression- Linear regression- Linear models, A bi-dimensional example, Linear Regression Robust regression with random sample consensus, Polynomial regression, Isotonic regression Logistic regression-Linear classification, Logistic regression, Implementation and Optimizations, Stochastic gradient descent algorithm.

Course Outcomes

CO1	Distinguish different learning based applications. (BTL2 : Understand)UNIT1
CO2	To apply different processing methods to prepare training data set for Machine Learning. (BTL3 : Apply) UNIT2
CO3	To design and implement supervised and unsupervised machine learning algorithm. (BTL3 : Apply)UNIT 3
CO4	To implement different learning models. (BTL3 : Apply) UNIT4
CO5	Learn Meta classifiers and deep learning concepts. (BTL2 : Understand)UNIT 5

Regression

- Regression in machine learning is a **supervised learning** technique .
- where the **primary goal is to predict a continuous numerical output (dependent variable)** based on one or more input features **(independent variables)**.

linear models in machine learning

- A **linear model** is a mathematical model that assumes a **linear relationship** between the input features (X) and the target output (y). In simple terms, it predicts the output as a **weighted sum of input feature**

- **s plus a bias term:**

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

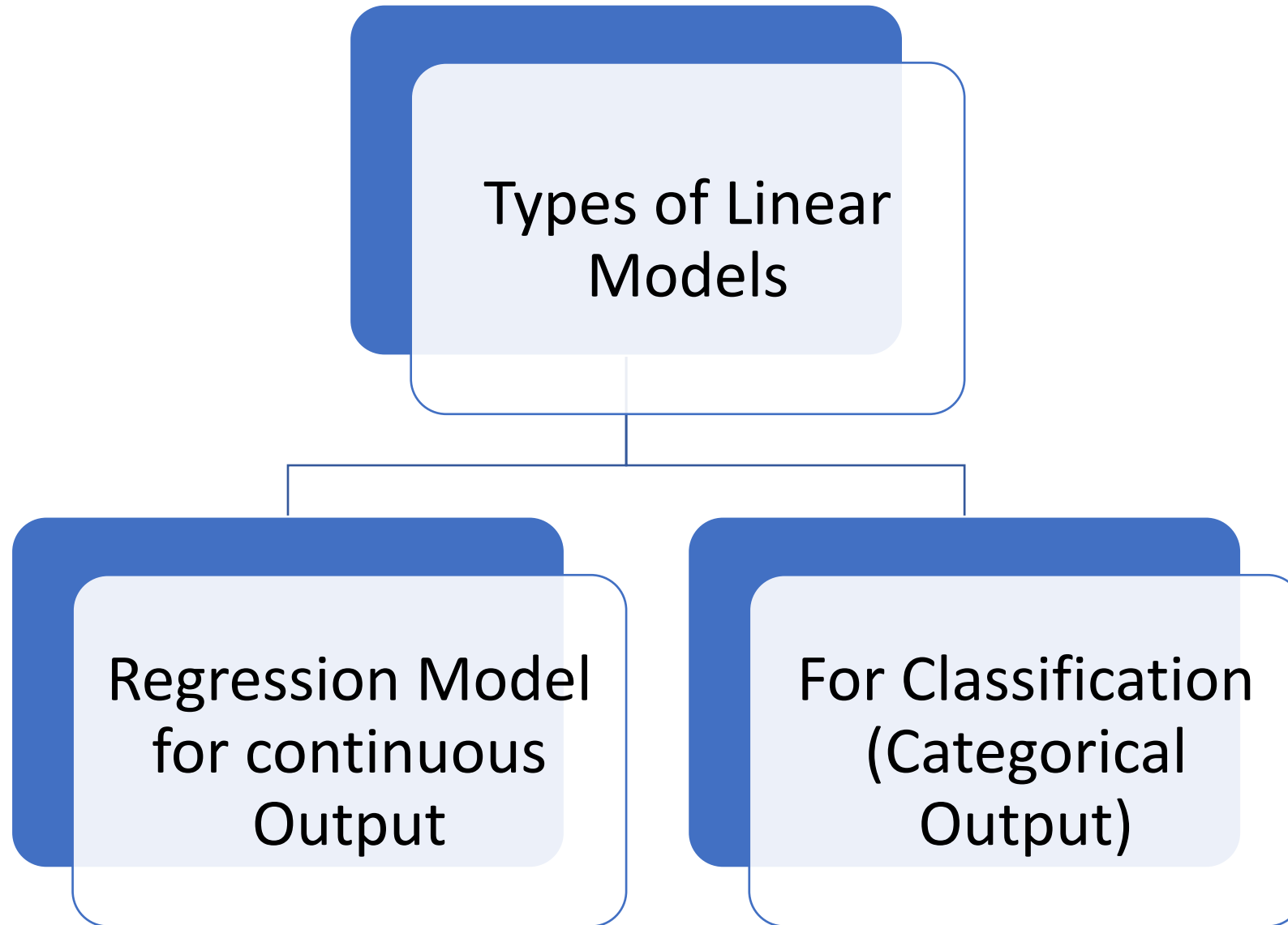
Where:

- w_i = weight (coefficient) for feature x_i
- b = bias/intercept
- n = number of features

Key idea: Changes in inputs cause proportional changes in the output (no curves, just straight lines or hyperplanes in higher dimensions).

linear models in machine learning

- Two crucial linear models in machine learning: **linear regression and logistic regression.**
- Linear regression is used for **regression tasks**, whereas logistic regression is a **classification algorithm.**
- **NOTE : These topics are covered during unit 1 in details. Please refer material accordingly**



Common Algorithms for Regression (Continuous Output)

- **Linear Regression:** Models a linear relationship between input and output.
- **Multiple Linear Regression :** Multiple input features, output is still continuous.
- **Polynomial Regression:** Models a non-linear relationship using polynomial functions.
- **Ridge and Lasso Regression:** Regularization techniques to prevent overfitting in linear models.
- **Support Vector Regression (SVR):** An extension of Support Vector Machines for regression tasks.
- **Decision Trees and Random Forests:** Can be adapted for regression problems.

Common Algorithms for Classification (Categorical Output)

- **Logistic Regression**

- Uses a sigmoid function to map outputs to probabilities for binary classification.

- **Multinomial Logistic Regression**

- Extension of logistic regression for multi-class problems.

- **Linear Discriminant Analysis (LDA)**

- Finds linear combinations of features that separate classes.

- **Perceptron**

- Early linear classifier, basis for neural networks.

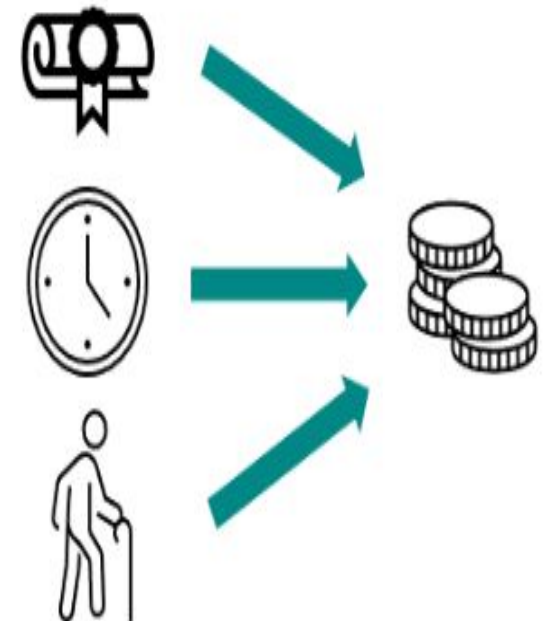
Linear Regression

- Linear regression analysis is used to create a model that describes the **relationship between a dependent variable and one or more independent variables.**
- Depending on whether there are one or more independent variables, a distinction is made between **simple and multiple linear regression** analysis.

Simple Linear Regression



Multiple Linear Regression

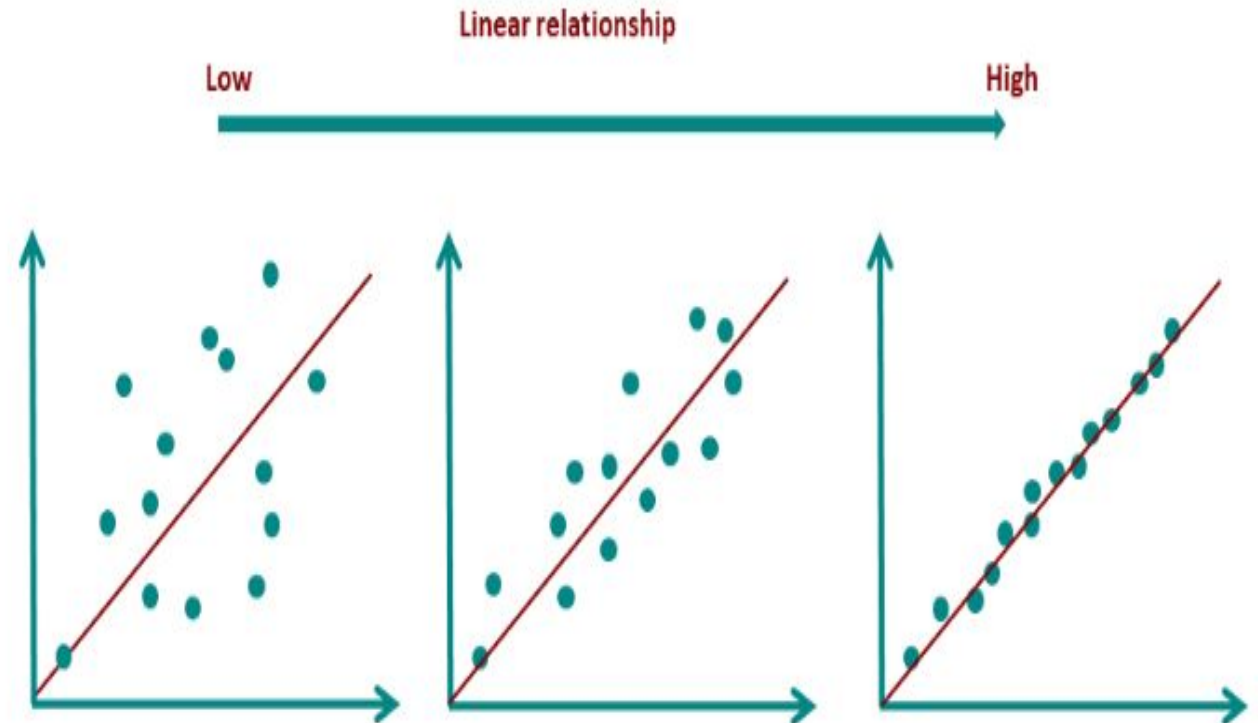


Example

- **Simple Linear Regression**
 - Does the **height** have an influence on the **weight** of a person?
- **Multiple Linear Regression**
 - Do the **height** and **gender** have an influence on the **weight** of a person?
- **Dependent variable**
- **Independent variables**

Simple Linear Regression

- The goal of a **simple linear regression** is to predict the value of a dependent variable based on an independent variable.
- The greater the linear relationship between the independent variable and the dependent variable, the more accurate is the prediction.
- The more the data points lie on a straight line.



Simple Linear Regression

- The task of simple linear regression is to exactly determine the straight line which best describes the linear relationship between the dependent and independent variable.
- In linear regression analysis, a straight line is drawn in the scatter plot. To determine this straight line, linear regression uses the **method of least squares**.
- The regression line can be described by the following equation:

$$\hat{y} = b \cdot x + a$$

Estimated dependent variable

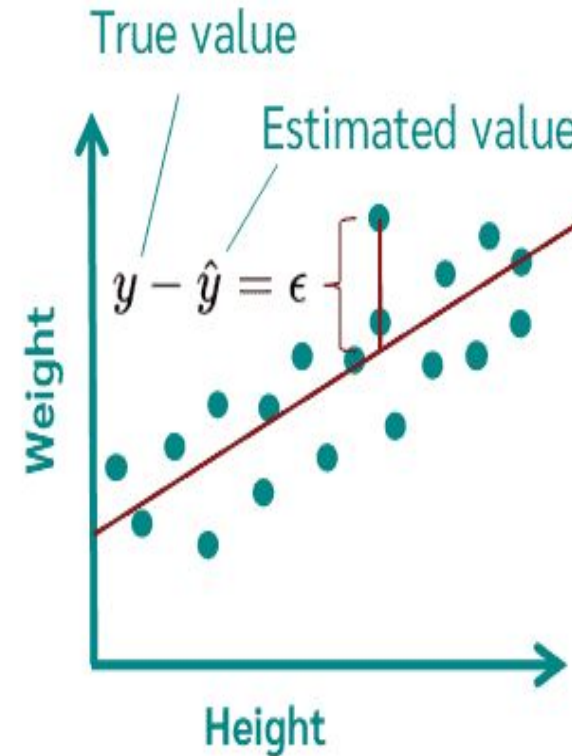
Slope

Independent variable

y intercept

Simple Linear Regression

- **Regression coefficients:**
- **a** : point of intersection with the y -axis.
- **b** : gradient of the straight line.
- $\hat{y}$ is the respective estimate of the y -value. This means that for each x -value the corresponding y -value is estimated. In our example, this means that the height of people is used to estimate their weight.



Error epsilon

$$y = b \cdot x + a + \boxed{\epsilon}$$

A bi dimensional example

- A bi-dimensional example in machine learning refers to a scenario where the **data being analyzed or modeled has two features or attributes**. This means each data point can be represented in a 2D space, often visualized on a scatter plot.

Example Scenario

- We want to **predict house price** using:
 - **Size of the house (m²)** → x_1
 - **Number of rooms** → x_2
- Output:
 - **Price in lakhs** → y

So necessary Linear Model will be



Linear Model Formula

The general form:

$$y = w_0 + w_1x_1 + w_2x_2$$

Where:

- w_0 = bias (intercept)
- w_1, w_2 = coefficients that tell us how much each feature affects the price

1. Sample Data



Size (x1)	Rooms (x2)	Price (y)
100	2	50
150	3	65
200	4	80
250	4	95
300	5	110

2. Model After Training



$$y = 20 + 0.25x_1 + 5x_2$$

Meaning:

- Each extra m² increases price by ₹0.25 lakh.
- Each extra room increases price by ₹5 lakh.

3. Prediction Example

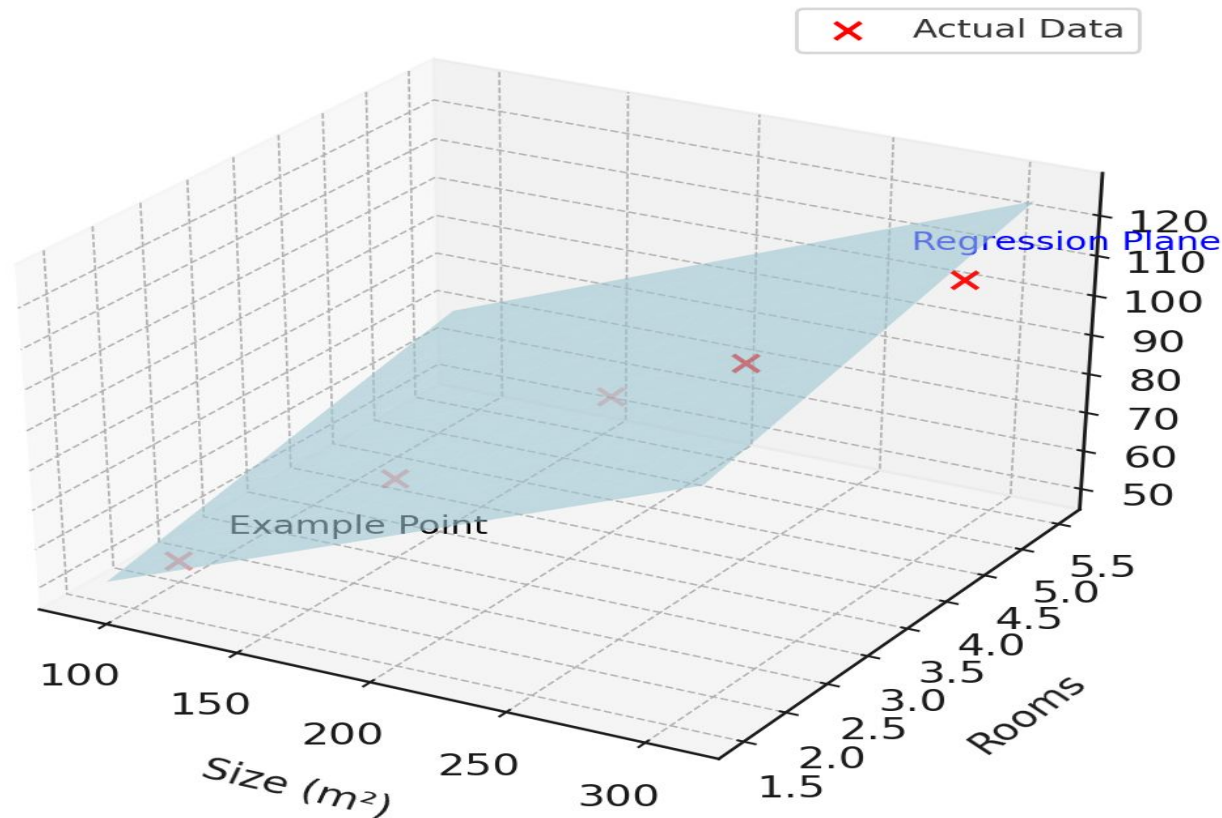


For a 200 m² house with 3 rooms:

$$y = 20 + 0.25(200) + 5(3) = 85 \text{ lakhs}$$

4. Predicted House price is 85 Lakhs for area 200 meter sq. with 3 rooms

Bi Dimensional Example



Red points → Actual house data

Blue surface → Regression plane predicted by the model

Example Point label shows one observation

Axes show **two features** (Size, Rooms) and the **target** (Price)

Why Bi-Dimensional Examples are Necessary

- Bi-dimensional examples in machine learning are especially **useful for learning, visualization, and concept understanding** because they give a **clear bridge between abstract math and real-world problems.**
- **Easy Visualization**
 - With **two input features**, we can plot them on a 2D plane (or 3D with the output) and show how a model works.
 - Helps learners **see** what a regression plane or decision boundary actually looks like.

Why Bi-Dimensional Examples are Necessary

- **Concept Simplification**

- Simplifies understanding of **multi-feature relationships** before moving to high-dimensional data (where visualization becomes impossible).
- Makes it easier to grasp core ML concepts like:
 - Linear regression
 - Overfitting vs underfitting
 - Regularization
 - Feature importance

- **Intuitive Model Interpretation**

- With two features, it's easy to explain **how each feature affects the prediction**.
- Useful for **demonstrating coefficient meaning** in linear models.

Real-World Use Cases for Bi-Dimensional Examples

1.Housing Price Estimation

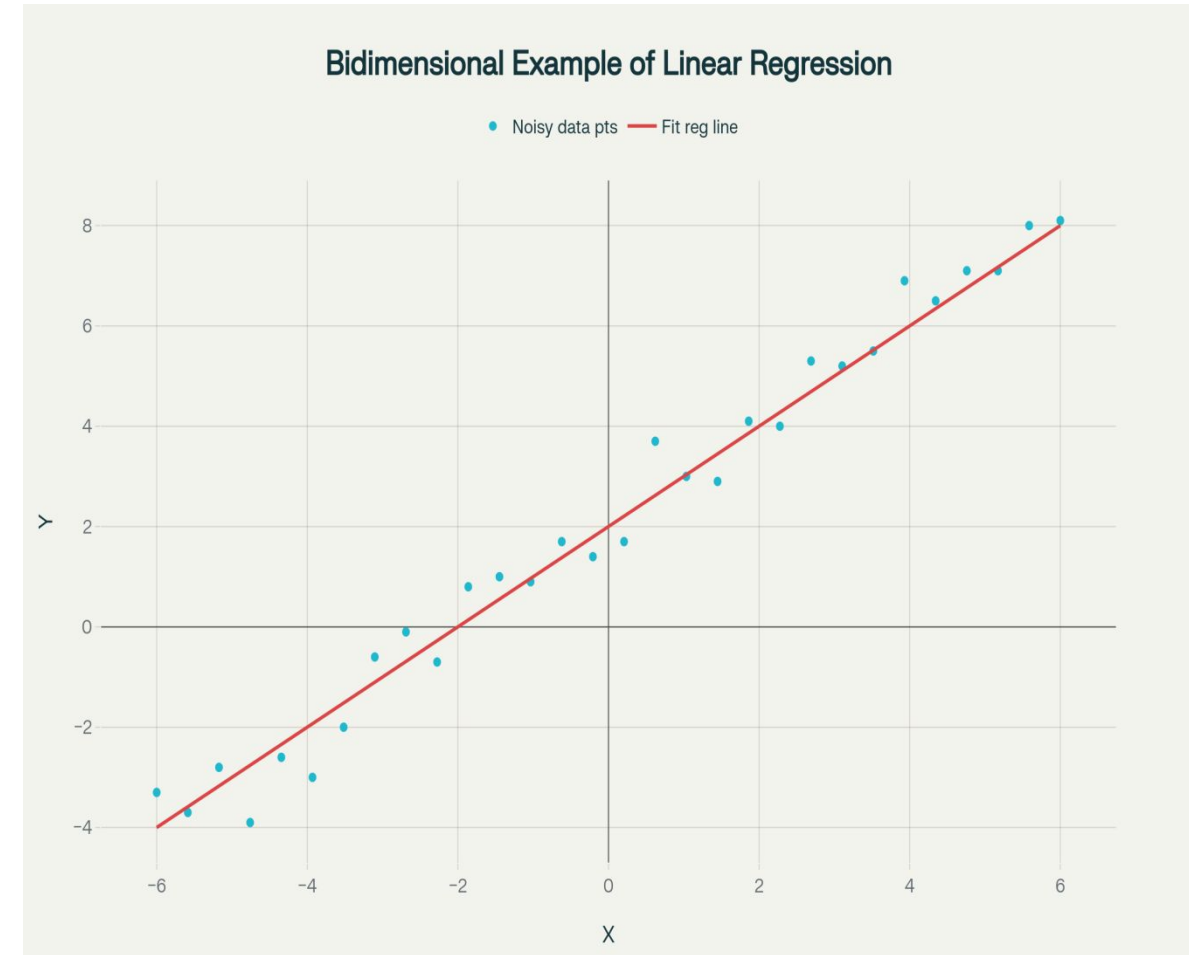
- Feature 1: Size of house (m²)
- Feature 2: Number of rooms
- Output: Price

2.Crop Yield Prediction

- Feature 1: Rainfall (mm)
- Feature 2: Fertilizer used (kg)
- Output: Yield (tons/hectare)

Bi Dimensional Example

- how linear regression works in two dimensions (i.e., with a single input variable X and an output variable Y) when there is some random noise in the data:
- Imagine you have points scattered along a line because of the ***underlying rule*** $y=x+2$,
 - but each point has a bit of random noise added, so they don't sit exactly on the line.
- **Linear regression tries to fit the best straight line through these noisy points.** The fitted line will be the rule that best summarizes the relationship.



Blue dots: represent the noisy data points (x,y) .

Red line: shows the regression line (best fit) calculated by linear regression.

Importance of Bidimensional Example

- It builds intuition for **how regression denoises data**: the fitted line is the best estimate, ignoring the random ups and downs (noise).
- **Shows mathematical simplicity**:
 - the loss function to minimize is $\sum (a + \beta x_i - y_i)^2$
 - the sum of squared vertical distances from the points to the line.
- **Scaling up**: We extend regression to higher dimensions (more features) and handle real datasets easily.

Random Regression

- Robust regression is a variation of regression techniques designed to deal with outliers or data contaminated with influential observations.
- It **minimizes the influence of outliers** by down-weighting them, often using iterative algorithms.
- The focus is often on **fitting a model while minimizing the influence of anomalies or noise by random sampling strategies**

Random Regression

- Advantages:
 - Provides better estimates when outliers are present.
 - Less restrictive assumptions about data distribution.
 - Better fit for the majority of data points.
- Disadvantages:
 - More complex to interpret and implement.
 - May result in loss of information if many points are down-weighted.
 - Requires careful handling and diagnostic checks.

Example Algorithm

Algorithm	Main Mechanism	Outlier Handling
Huber	Weighted Least Squares	Down-weights large residuals
RANSAC	Random subset + consensus inliers	Ignores outliers completely
Theil-Sen	Median of regression slopes	Resistant to extreme values

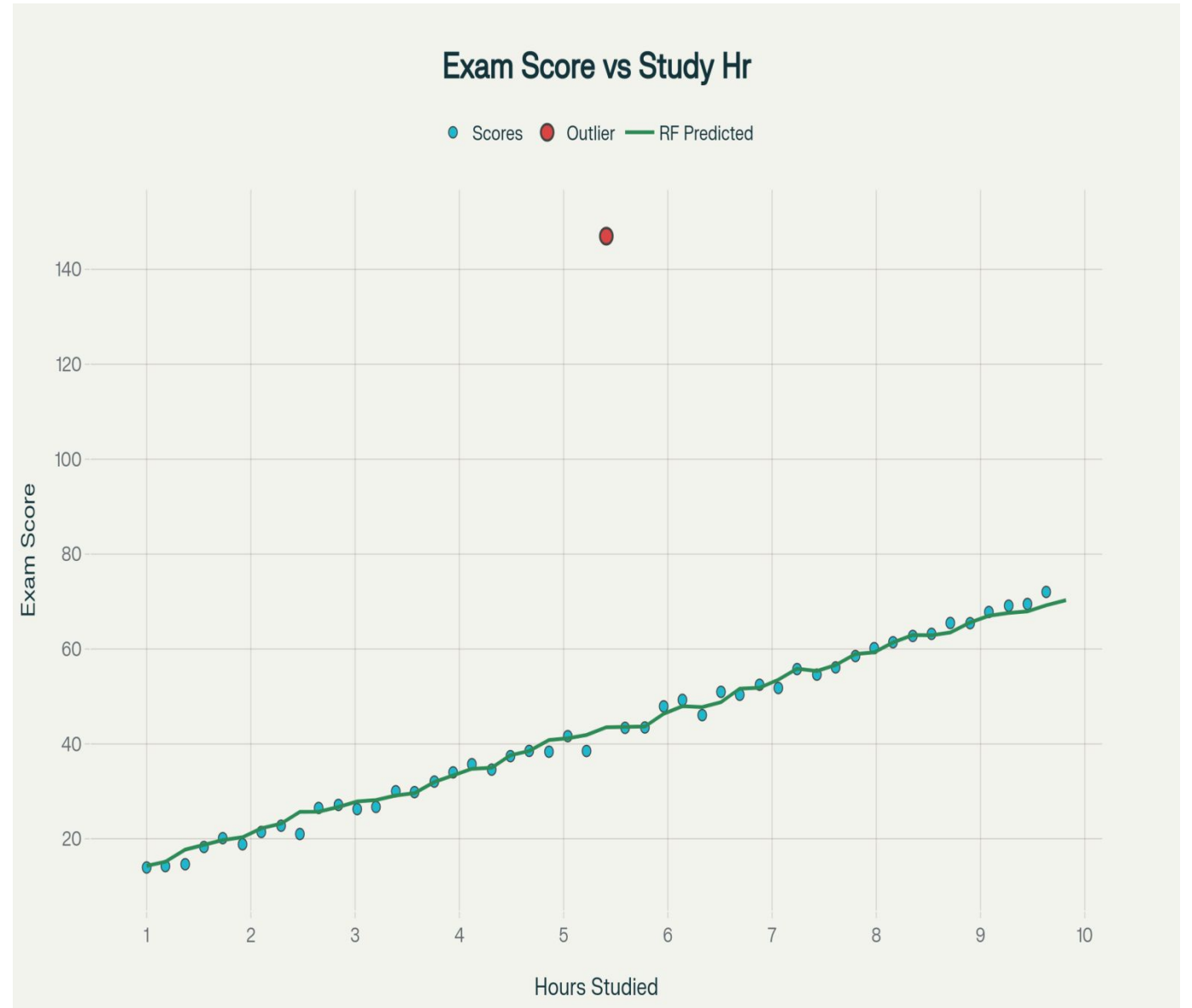
Example Robust regression

- Imagine your exam score dataset contains errors—maybe one student's score was entered incorrectly, resulting in an outlier. Regular linear regression might skew heavily due to this outlier. Robust regression, however, detects and assigns less weight to that data point, providing a model that better represents the majority of the data.

- Let's imagine a scenario where exam scores depend largely on hours studied, but there's an error in the dataset for one student (outlier):
- Typical relationship: Most students follow a trend:
 - *Exam Score = 5 × Hours Studied + 10 + random noise.*
- Outlier: One student's score is mistakenly entered as 150—much higher than expected for their hours studied.

Visualization

- Blue dots: Regular student exam scores (with some noise).
- Red dot: Outlier, representing an erroneous exam score.
- Green line: Predictions from a Random Forest regression fitted to the data.



Random Sample Consensus (RANSAC)

- It is an iterative method used for fitting models to datasets containing a significant proportion of outliers.
- **The main idea is: Randomly pick a subset of data points to fit a model.**
- Verify how many points in the overall dataset fit this model within a tolerance (these are "inliers").
- Repeat the process to find the model with the best consensus among inliers
- RANSAC is often used in combination with linear regression **for computer vision, geospatial modeling**, and any scenario where outliers can significantly skew results. Robust regression provides an alternative when you want your model to learn from all data but reduce the impact of erroneous or influential observations. Combining RANSAC with robust regression gives extra outlier resistance for model fitting in machine learning pipelines

Random Regression with RANSAC

- RANSAC works by random sampling subsets and choosing the best fit with maximum consensus.

Working Steps of RANSAC Regression

1. Randomly select a small subset of the original data points (minimum number required to fit the model).
2. Fit the model (e.g., a line for linear regression) to this random subset.
3. Test all other data points against the fitted model:
 - Points that fit within a certain error threshold are classified as *inliers*.
 - Points that don't fit well are *outliers*.
4. Store the model and its consensus set (inliers).
5. Repeat steps 1-4 for a fixed number of iterations or until a model with sufficient inliers is found.
6. Choose the model with the largest consensus set.
7. Optionally, refit the model to all inliers for improved accuracy.
8. This way, RANSAC effectively ignores outliers and fits the model only on good-quality data points, making it robust.

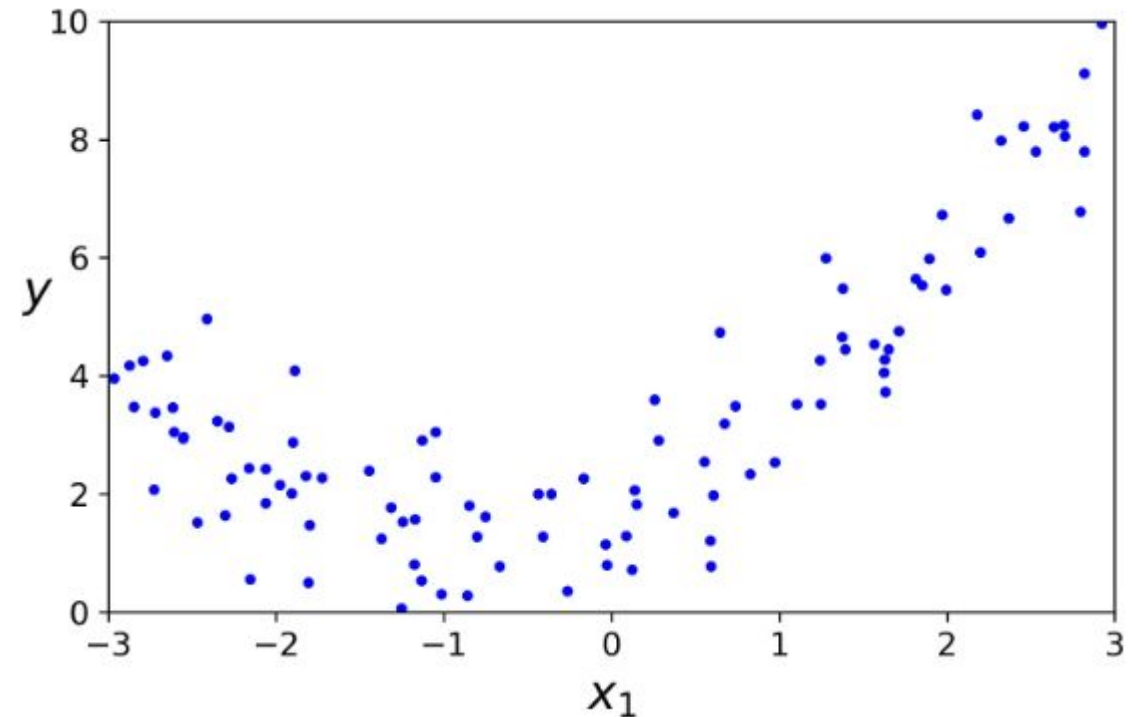
Polynomial regression

- Polynomial regression is an essential extension of linear regression used to model **non-linear relationships** in data.
- where the relationship between the **independent variable (x) and the dependent variable (y) is modelled as an n th degree polynomial**. It is useful when the data exhibits a non-linear relationship allowing the model to fit a curve to the data.

Polynomial regression

- Unlike linear regression, which assumes a straight-line relationship between variables, polynomial regression captures curved patterns by adding higher-degree polynomial terms. The general equation takes the form:

$$y = b_0 + b_1x + b_2x^2 + \dots + b_nx^n + \epsilon$$



In this equation, x^n represents polynomial terms, b_n are coefficients, and ϵ accounts for error.

With more polynomial terms, the model becomes more flexible in capturing **non-linear trends**.

Aspect	Linear Regression	Polynomial Regression
Definition	Models the relationship between dependent (Y) and independent variable(s) (X) as a straight line.	Models the relationship between dependent (Y) and independent variable(s) (X) as an nth degree polynomial curve.
Equation	$Y = \beta_0 + \beta_1 X + \epsilon$	$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \dots + \beta_n X^n + \epsilon$
Nature of Relationship	Assumes a linear relationship between X and Y.	Captures non-linear relationships by introducing polynomial terms.
Flexibility	Less flexible, only fits straight lines.	More flexible, can fit curves depending on the degree of polynomial.
Interpretability	Easy to interpret coefficients.	Becomes harder to interpret as polynomial degree increases.
Overfitting	Less prone to overfitting (simple model).	High degree polynomials may lead to overfitting.
Computational Complexity	Low computational cost.	Higher computational cost with increasing degree.
Use Cases	- Predicting salary vs. years of experience (if roughly linear) - House prices with one linear feature	- Modeling growth patterns (population, disease spread) - Non-linear trends like learning curves, demand prediction
Assumptions	Linear relationship, homoscedasticity, independence of errors, normality.	Same assumptions, but allows non-linearity through transformation.
Example	Predicting student scores based on study hours (linear).	Predicting salary growth where income rises non-linearly with experience.

Case Study: Predicting Ice Cream Sales using Polynomial Regression

- The **Ice Cream Selling dataset** is a simple yet powerful resource for beginners in machine learning. It contains two columns:
- **Temperature (°C)** → independent variable (X)
- **Units of Ice Cream Sold** → dependent variable (y)
- This dataset captures the natural relationship between temperature and ice cream sales: as temperature rises, more ice cream is sold. However, the growth is **non-linear** (sales accelerate more at higher temperatures).
- This makes it an ideal example to apply **Polynomial Regression**, which extends Linear Regression by adding higher-order terms to better capture non-linear patterns.

Objective

- To **predict ice cream sales** based on temperature.
- To compare **Linear Regression vs. Polynomial Regression**.
- To visualize and evaluate how polynomial regression captures non-linear growth in sales.

Dataset Overview

Temp (°C)	Sales (₹k)
10	15.0
12	18.4
14	22.4
16	26.8
18	31.6
20	37.0
22	42.8
24	49.2
26	56.0
28	63.2
30	71.0
32	79.2

Step-by-step solution

1) Define target and feature

- **Feature (X):** Temperature (°C)
- **Target (y):** Sales (₹k)

2) Train/test split

- Use 80/20 split to evaluate generalization.

3) Baseline (linear regression)

$$\text{Fit } y = \beta_0 + \beta_1 x.$$

- Expect underfit at high temperatures (systematic residuals).

4) Create polynomial features

- For degree $d=2$: $[x, x^2]$
- Optionally try $d=3, 4, \dots$ but avoid overfitting.
- (Tip) Standardize features *after* expansion or use a Pipeline.

Step-by-step solution

5) Model fitting

- Fit linear model on expanded features:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

6) Model selection

- Use **cross-validation** to pick degree (e.g., from {1,2,3}).
- Prefer lowest test RMSE / highest test R^2 .
- If degrees > 2, consider **Ridge** to stabilize coefficients.

7) Evaluate on test set

- Report **RMSE, MAE, R^2** .
- Plot predictions vs. actuals or residuals to check fit (curve should match acceleration at high temps).

Step-by-step solution

8) Interpret & use

- Positive β_2 means **accelerating growth** with temperature.
- Use the model to predict sales for, say, 27°C.
- **Caution:** Don't extrapolate far outside the observed temp range.

Isotonic Regression

- **Isotonic Regression** is a **non-parametric regression technique** that fits a **monotonically increasing (or decreasing)** function to the data.
- "Isotonic" = always increasing.
- "Antitonic" = always decreasing.
- Unlike linear or polynomial regression, it **does not assume a specific functional form** (like a line or curve). Instead, it ensures that predictions **preserve the order**

if $x_1 < x_2$, then predicted $y_1 \leq y_2$ (for isotonic).

Isotonic Regression

- **Monotonicity constraint:** The fitted function is always non-decreasing (or non-increasing).
- **Non-parametric:** No fixed functional form (flexible shape).
- **Stepwise function:** The fitted curve is piecewise constant or piecewise linear.
- **Handles noisy data:** Smooths data while keeping monotonicity.

Linear Classification

- In **Machine Learning, Linear Classification** is a technique where a **linear function (straight line, plane, or hyperplane)** is used to separate data points into different classes.
- The decision rule is based on:
 - $y = w^T x + b$
 - x = input features
 - w = weights
 - b = bias (intercept)
- If $y > 0$, predict class **1**, else predict class **0**.
This creates a **linear decision boundary** in the feature space.

Uses of Linear Classification

- **Binary classification:** spam vs not spam, pass vs fail, disease vs no disease.
- **Multi-class classification:** digit recognition (0–9), sentiment analysis.
- **High-dimensional data problems:** text classification, document categorization.
- **Baseline models:** fast, interpretable models before trying complex ones.

Advantage

1. Simple & easy to implement.
2. Computationally efficient, works well with large datasets.
3. Easy to interpret (weights show feature importance).
4. Works well when classes are **linearly separable**.
5. Good for high-dimensional sparse data (e.g., text, bag-of-words)

Disadvantages

- Cannot handle non-linear relationships (fails if classes overlap in curves).
- Sensitive to **outliers** (decision boundary can shift).
- May **underfit** complex datasets.
- Needs **feature scaling/engineering** for good performance.
- Not effective when data has complex interactions unless extended with kernel

Algorithms Used for Linear Classification

1. **Logistic Regression**

- Uses sigmoid function to map outputs to probabilities.
- Widely used for binary and multi-class problems.

2. **Linear Discriminant Analysis (LDA)**

- Assumes features are normally distributed.
- Projects data to maximize class separability.

3. **Linear Support Vector Machine (Linear SVM)**

- Finds the **maximum-margin hyperplane** that best separates classes.

4. **Perceptron**

- One of the earliest neural network models.
- Iteratively adjusts weights to find a separating hyperplane.

5. **Ridge Classifier**

- Uses **linear regression with regularization** but outputs discrete classes.

6. **Naïve Bayes (log-space gives linear boundaries)**

- Though probabilistic, often results in linear decision boundaries.

Overfitting (High Variance Problem)

- **Definition:**

Overfitting happens when a model learns **too much detail** from the training data — including **noise and random fluctuations** — so it performs very well on training data but **poorly on unseen/test data**.

- Model is **too complex** (too many parameters, high-degree polynomial, deep tree).
- Memorizes training data instead of generalizing patterns.

- **Example:**

- Suppose you want to predict **student exam scores** based on study hours.
- You train a **10th-degree polynomial regression**.
- It will **fit training points perfectly**, but on new students, predictions will be inaccurate.

Under fitting (High Bias Problem)

- Under fitting happens when a model is **too simple** and fails to capture the underlying pattern in the data. It performs poorly both on **training** and **test** data.
- Model is **not complex enough** (linear model on nonlinear data).
- Ignores important relationships in features.
- **Example:**
 - Same exam score vs. study hours problem.
 - If you fit a **straight line (Linear Regression)** but the actual trend is quadratic (curved), the model will be inaccurate.

How to Handle Them

- **Avoid Overfitting:**

- Use **regularization** (Lasso, Ridge).
- Simplify the model.
- Collect more training data.
- Use **cross-validation**.

- **Avoid Underfitting:**

- Use a more **complex model** (e.g., decision tree instead of linear regression).
- Add more relevant features.
- Reduce regularization (don't restrict model too much).

Summary

Aspect	Underfitting	Overfitting
Definition	Model too simple, misses patterns	Model too complex, memorizes noise
Bias	High	Low
Variance	Low	High
Training Accuracy	Low	High
Test Accuracy	Low	Low
Solution	Increase complexity, add features	Regularize, simplify, more data

Optimizations

- Optimization is the process of making something—such as a system, design, decision, or algorithm—as effective, efficient, or functional as possible given a set of conditions.
- The 3 Pillars of Any Optimization Problem :
 1. **Objective (The Goal):** What you are trying to **maximize** (e.g., profit, speed, accuracy, efficiency) or **minimize** (e.g., cost, error, time, energy usage).
 2. **Variables (The Levers):** The factors or parameters you can adjust to change the outcome.
 3. **Constraints (The Boundaries):** The strict limits or rules you must work within (e.g., budget, time limits, physical space, legal regulations).

Optimizations

Maximization vs. Minimization

- **Minimization:** Reducing unwanted outcomes.

$$\min_x \text{Cost}(x) \quad \text{or} \quad \min_{\theta} \text{Loss}(\theta)$$

- **Maximization:** Boosting desired outcomes.

$$\max_x \text{Efficiency}(x) \quad \text{or} \quad \max_x \text{Profit}(x)$$

Optimizations

- In **regression**, optimization refers to the mathematical process of finding the **best set of model parameters (coefficients/weights)** that minimize (or sometimes maximize) an **objective function**.
- Optimization = adjusting weights of the regression equation until the model gives the **most accurate predictions**.
- Optimization in regression analysis focuses on minimizing a **cost function**, such as Mean Squared Error (MSE), to find the model coefficients that best fit the data and accurately predict outcomes, using techniques like regularization for high-dimensional datasets.

Cost function /loss function

- In machine learning, **a cost function is a mathematical metric that quantifies the error or "cost" between a model's predicted outcomes and the actual.**
- **Its primary purpose is to measure how poorly a model is performing, and the core goal during model training is to minimize this cost function.**
- It's like a **report card** for the model:
 - High cost → model is performing poorly.
 - Low cost → model is performing well.
- Common examples:
 - **Mean Squared Error (MSE)** in regression.
 - **Cross-Entropy / Log Loss** in classification.

Example

- **Real-Time Example: Food Delivery App (Swiggy/Zomato/Ubereats)**
- You want to build a regression model to predict **delivery time** (in minutes) based on:
 - Distance from restaurant
 - Traffic conditions
 - Weather
- **Test your model on 3 deliveries**

Delivery	Actual Time (min)	Predicted Time (min)	Error (Predicted - Actual)
1	30	40	+10
2	25	20	-5
3	45	50	+5

• Step 1: Calculate Errors

- Error = difference between **predicted** and **actual** times.
But errors can be **positive or negative**, so we square them to avoid cancellation.

$$Error^2 = (Predicted - Actual)^2$$

- Delivery 1 $\rightarrow (40 - 30)^2 = 100$
- Delivery 2 $\rightarrow (20 - 25)^2 = 25$
- Delivery 3 $\rightarrow (50 - 45)^2 = 25$

• Step 2: Compute Cost Function (MSE)

- The **Mean Squared Error (MSE)** is the average of squared errors:

$$MSE = \frac{100 + 25 + 25}{3} = 50$$

Cost Function = 50

- **Conclusion** : On average, our model is making errors equivalent to $\sqrt{50} \approx 7$ **minutes** in delivery time.
- Lower MSE means better accuracy.

- **Why is this Useful?**

- If the cost is high → model is bad at predicting.
- Optimization algorithms (like Gradient Descent) adjust the weights to **minimize this cost**.
- Eventually, the model will predict closer to reality (like 32 mins instead of 50).

- **Summary:**

- The **cost function** is like the **customer complaints meter**
- More complaints (higher errors) → high cost.
- Fewer complaints (accurate delivery time predictions) → low cost.

Why is Optimization Needed?

- Without optimization, regression models wouldn't know how to find the best-fitting line or curve for the data.
- Helps the model **reduce errors** between predictions and actual values.
- Prevents Underfitting (too simple) or overfitting (too complex).
- Improves **generalization** → better predictions on unseen data.
- **How Optimization Improves ML Models**
 - **Linear Regression** → Finds best line to fit data.
 - **Logistic Regression** → Finds decision boundary for classification.
 - **Neural Networks** → Optimization tunes millions of weights.
 - **Regularized Regression (Lasso/Ridge)** → Prevents overfitting, improves performance on unseen data

Common Optimization Technique in Regression

- **Normal Equation** (closed-form solution) – directly computes best weights (only feasible for small datasets).
- **Gradient Descent (GD)** – iterative method that updates weights step by step until the loss function is minimized.
- Variants: **Batch GD, Stochastic GD (SGD), Mini-batch GD.**
- **Regularization techniques (L1, L2)** – add penalty terms to prevent overfitting.

Gradient Descent (GD)

- Gradient Descent is an iterative optimization algorithm used to find the minimum of a function.
- **Gradient Descent is an algorithm** that is used to optimize the cost function or the error of the model. It is used to find the minimum value of error possible in your model.
- In machine learning, it minimizes a loss or cost function—representing model error—by tweaking parameters (**weights and biases**) in the direction of the steepest downward slope until error is as small as possible.

Gradient Descent (GD)

What is Weight (w)?

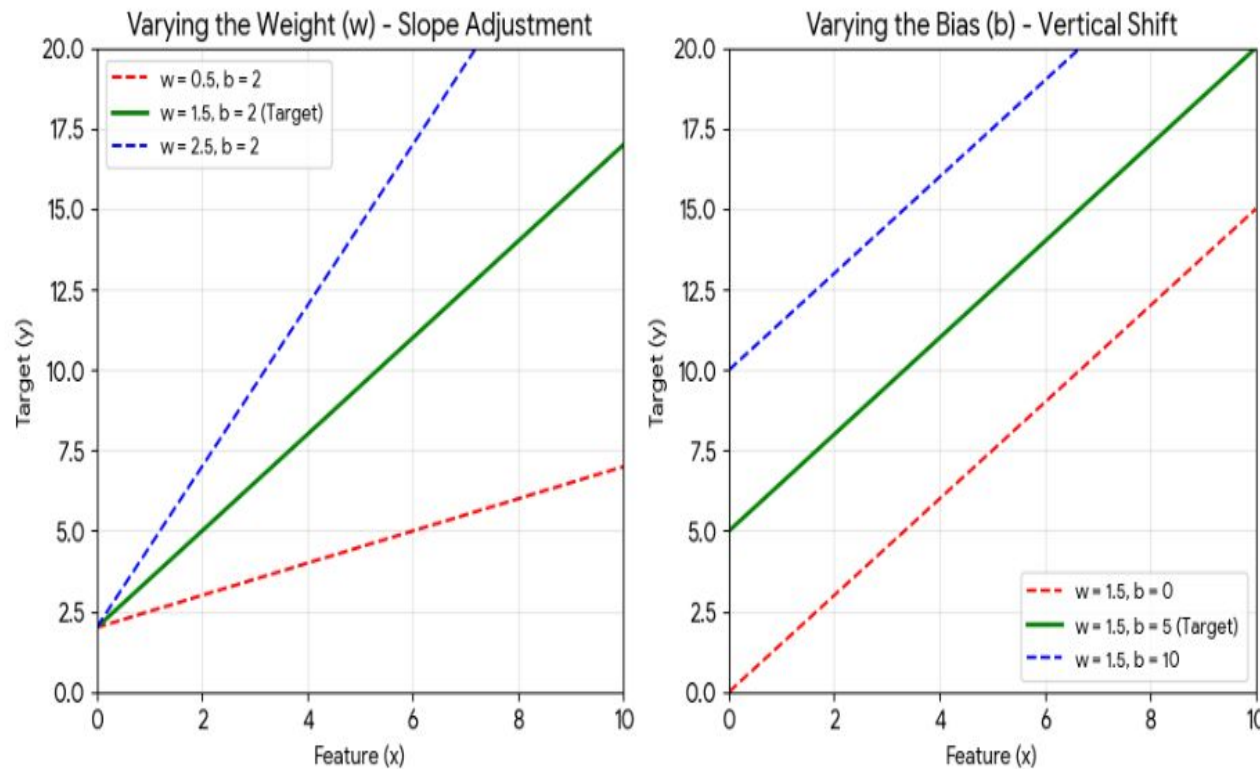
The weight controls the strength or importance of a specific input feature. It determines how much a change in the input affects the final prediction.

- The Math: It multiplies the input feature ($w \cdot x + b$).
- Analogy: If you are predicting house prices, the "number of bedrooms" feature will have a high positive weight because it heavily increases the price. The "color of the front door" feature will have a weight close to zero because it barely matters.
- Weight determines the slope of the relationship between your input and output.

.What is Bias (b)?

- The bias represents the baseline value or starting point of the prediction, completely independent of the input features.
- The Math: It is added at the very end of the calculation ($w \cdot x + b$).
- Analogy: Even if a house has 0 bedrooms and 0 square feet (an empty lot), it still costs a baseline amount of money just for the land. That base price is the bias.
- Bias shifts your entire prediction line up or down. Without a bias, your prediction line would be forced to go through $((0,0))$, which means a house with 0 square feet would have to be predicted at exactly \$0.

Gradient Descent (GD)



The Complete Equation

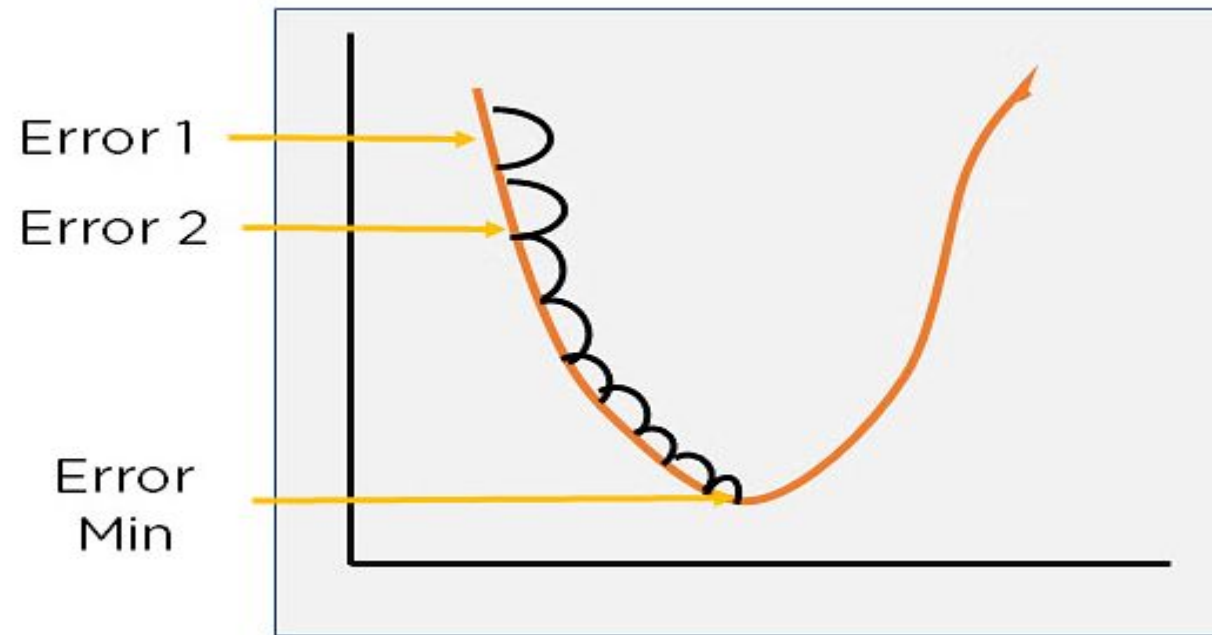
When combined, they form the classic equation for a straight line:

$$\hat{y} = w \cdot x + b$$

During **Gradient Descent**, the algorithm calculates the error of this line. If the line is too flat, it changes the **weight** to tilt it. If the line is too low, it changes the **bias** to lift it up—continuously adjusting both until the line fits the data points perfectly.

Gradient Descent (GD)

- In gradient descent, you find the error in your model for different values of input variables. This is repeated, and soon you see that the error values keep getting smaller and smaller. Soon you'll arrive at the values for variables when the error is the least, and the cost function is optimized.



Example

let's look at a simple **Linear Regression** case study using a dataset of **two houses**.

Our goal is to find the best line ($\hat{y} = w \cdot x + b$) that minimizes the total prediction error.

Our Dataset

- **House 1:** Size (x_1) = **1**, Price (y_1) = **2**
- **House 2:** Size (x_2) = **3**, Price (y_2) = **4**

Example

Step 1: Initialize Parameters & Hyperparameters

We start with random guesses for our weight and bias, and set a small step size.

- Weight (w) = 0.5
- Bias (b) = 0.0
- Learning Rate (α) = 0.1

Example

Step 2: Make Predictions & Calculate Total Cost

We calculate predictions for all data points and evaluate the model using Mean Squared Error (MSE).

$$\text{MSE } J(w, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

- **House 1 Prediction:** $\hat{y}_1 = (0.5 \cdot 1) + 0 = 0.5 \rightarrow \text{Error: } 0.5 - 2 = -1.5$
- **House 2 Prediction:** $\hat{y}_2 = (0.5 \cdot 3) + 0 = 1.5 \rightarrow \text{Error: } 1.5 - 4 = -2.5$
- **Total Cost (J):** $\frac{1}{2} [(-1.5)^2 + (-2.5)^2] = \frac{1}{2} [2.25 + 6.25] = 4.25$

Example

Step 3: Compute the Gradients

We calculate the average gradient across all data points to see which way to turn the knobs.

$$\frac{\partial J}{\partial w} = \frac{2}{n} \sum (\hat{y}_i - y_i) \cdot x_i \quad \text{and} \quad \frac{\partial J}{\partial b} = \frac{2}{n} \sum (\hat{y}_i - y_i)$$

- Gradient for w :

$$\frac{\partial J}{\partial w} = \frac{2}{2} [(-1.5 \cdot 1) + (-2.5 \cdot 3)] = [-1.5 - 7.5] = \mathbf{-9.0}$$

- Gradient for b :

$$\frac{\partial J}{\partial b} = \frac{2}{2} [-1.5 + (-2.5)] = \mathbf{-4.0}$$

Example

Step 4: Update Parameters Simultaneously

- New Weight (w):

$$w := w - (\alpha \cdot \frac{\partial J}{\partial w}) = 0.5 - (0.1 \cdot -9.0) = 0.5 + 0.9 = \mathbf{1.4}$$

- New Bias (b):

$$b := b - (\alpha \cdot \frac{\partial J}{\partial b}) = 0.0 - (0.1 \cdot -4.0) = 0.0 + 0.4 = \mathbf{0.4}$$

Example

Step 5: Iterate and Check Convergence

We run the loop again using our updated line: $\hat{y} = 1.4x + 0.4$. Let's see if our error dropped.

- **House 1 New Prediction:** $\hat{y}_1 = (1.4 \cdot 1) + 0.4 = 1.8 \rightarrow$ New Error: $1.8 - 2 = -0.2$
- **House 2 New Prediction:** $\hat{y}_2 = (1.4 \cdot 3) + 0.4 = 4.6 \rightarrow$ New Error: $4.6 - 4 = +0.6$
- **New Total Cost (J):** $\frac{1}{2}[(-0.2)^2 + (0.6)^2] = \frac{1}{2}[0.04 + 0.36] = \mathbf{0.20}$

Stochastic Gradient Descent (SGD)

- **Stochastic Gradient Descent (SGD)** is an optimization algorithm used in **machine learning and deep learning** to minimize the **cost (loss) function** and update model parameters (weights).
- "Gradient Descent" = process of finding the **minimum of a function** by moving in the direction of steepest descent (negative gradient).
- "Stochastic" = instead of using the **entire dataset** at once, it updates parameters using **one random sample (or a small batch)** at a time.
- This makes SGD **faster** and **more scalable** for large datasets compared to traditional Gradient Descent.

Working of the Algorithm

- **Step 1: Initialize Parameters**

- Start with random weights (w) and bias (b).

- **Step 2: Pick a Random Sample**

- Instead of using the whole dataset, pick **one data point** (or a mini-batch).

- **Step 3: Compute Gradient**

- Calculate the **error** between prediction and actual value for that sample.

Then compute the **gradient of the loss function** with respect to weights.

- **Step 4: Update Parameters**

- Update weights using:

$$w := w - \eta \cdot \nabla L(w)$$

where:

- w = weight
- η = learning rate (step size)
- $\nabla L(w)$ = gradient of loss function

- **Step 5: Repeat**

- Repeat steps 2–4 for each sample until convergence
 - **Convergence** means the process where the algorithm gradually **approaches the minimum value of the cost (loss) function**.
 - The weights stop changing significantly.
 - The loss becomes stable (not reducing further).
 - The model has “learned” the optimal (or near-optimal) parameters.

Exercise

- How to implement **Stochastic Gradient Descent (SGD)** for **Handwritten Digit Recognition (MNIST dataset)**.

References

1. Giuseppe Bonaccorso, "Machine Learning Algorithms", Packt Publishing Limited, ISBN10: 1785889621, ISBN-13: 978-1785889622.
2. Ethem Alpaydin," Introduction to Machine Learning", PHI 2nd Edition-2013, ISBN 978-0-262-01243-0
3. 2. Peter Flach, "Machine Learning: The Art and Science of Algorithms that Make
4. Online Resources: NPTEL /SWAYAM lectures,youtube,springboard